	ORIGINATE DATE	EDIT DATE	DOCUMENT-REV. NUM.	PAGE
	7 May, 2001	4 September, 2015	GEN-CXXXXX-REVA	1 of 9

Author: Laurent Lefebvre

Issue To:	**Copy No:**

R400 Sequencer Specification

SEQ

Version 0.1

Overview: This is an architectural specification for the R400 Sequencer block (SEQ). It provides an overview of the required capabilities and expected uses of the block. It also describes the block interfaces, internal sub-blocks, and provides internal state diagrams.

AUTOMATICALLY UPDATED FIELDS:
Document Location: D:\Perforce\r400\arch\doc\gfx\MC\R400 MemCtl.doc
Current Intranet Search Title: R400 Memory Controller Architectural Specification

APPROVALS	
Name/Dept	Signature/Date

Remarks:

Table Of Contents

Revision Changes:

Rev 0.1 (Laurent Lefebvre) Date: May 7, 2001	First draft.

1. Overview

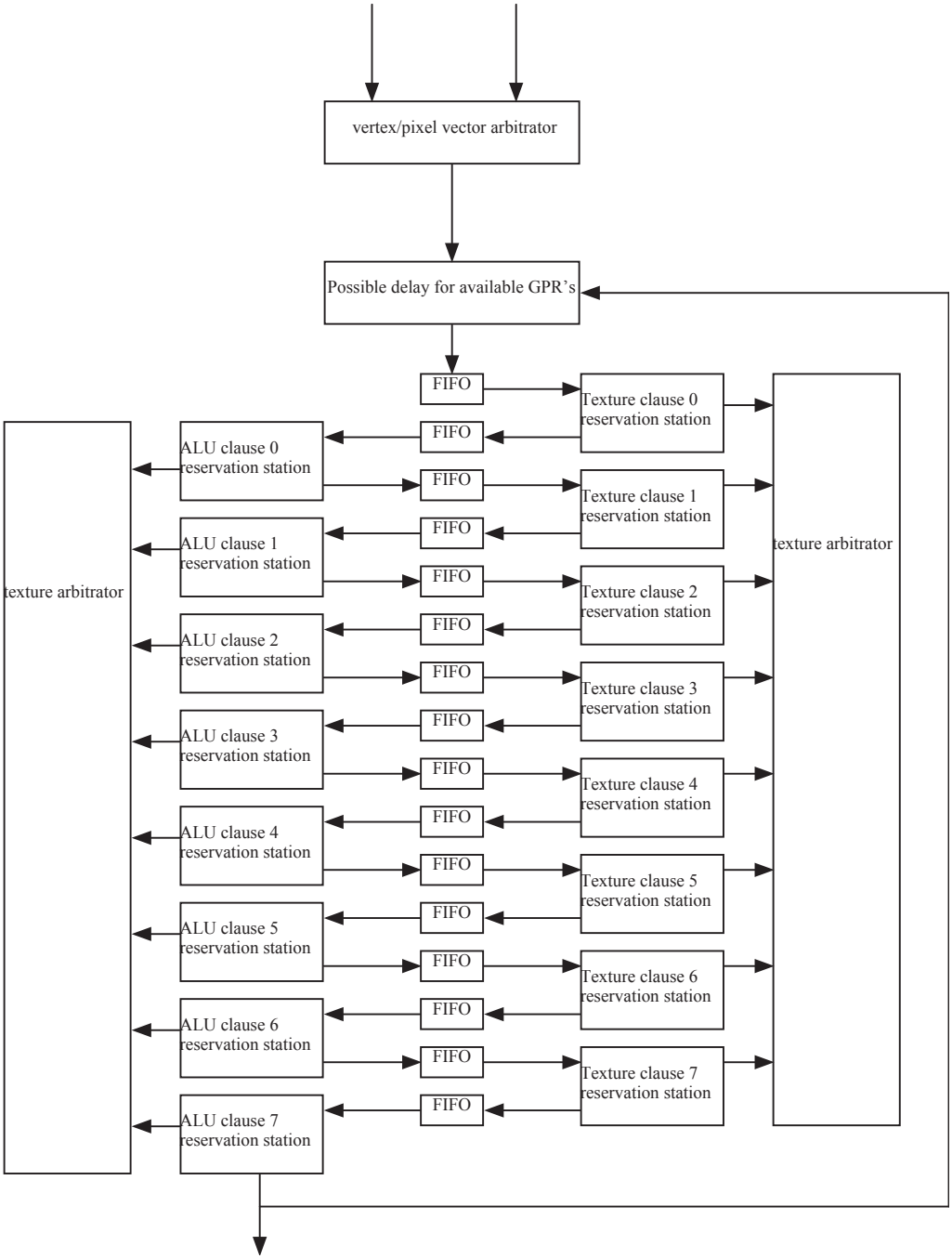
The sequencer first arbitrates between vectors of 16 vertices that arrive directly from primitive assembly and vectors of 8 quads (32 pixels) that are generated in the raster engine.

The vertex or pixel program specifies how many GPR's it needs to execute. The sequencer will not start the next vector until the needed space is available.

The sequencer is based on the R300 design. It chooses an ALU clause and a texture clause to execute, and execute all of the instructions in a clause before looking for a new clause of the same type. Each vector will have eight texture and eight alu clauses, but clauses do not need to contain instructions. A vector of pixels or vertices ping-pongs along the sequencer FIFO, bouncing from texture reservation station to alu reservation station. A FIFO exists between each reservation stage, holding up vectors until the vector currently occupying a reservation station has left. A vector at a reservation station can be chosen to execute. The sequencer looks at all eight alu reservation stations to choose an alu clause to execute and all eight texture stations to choose a texture clause to execute. The arbitrator will give priority to clauses/reservation stations closer to the top of the pipeline. It will not execute an alu clause until the texture fetches initiated by the previous texture clause have completed.

To support the shader pipe the raster engine also contains the shader instruction cache and constant store.

1.1 Top Level Block Diagram



The rasterizer always checks the vertices FIFO first and if allowed by the sequencer sends the data to the shader. If the vertex FIFO is empty then, the rasterizer takes the first entry of the pixel FIFO (a vector of 32 pixels) and sends it to the interpolators. Then the sequencer takes control of the packet.

On receipt of a packet, the input state machine (not pictured but just before the first FIFO) allocated enough space in the registers to store the interpolatoted values and temporaries. Following this, the input state machine stacks the packet in the first FIFO.

On receipt of a command, the level 0 texture machine issues a texure request and corresponding register address for the texture address (ta). A small command (tcmd) is passed to the texture system identifying the current level number

(0) as well as the register set being used. One texture request is sent every 4 clocks causing the texturing of four 2x2s worth of data.

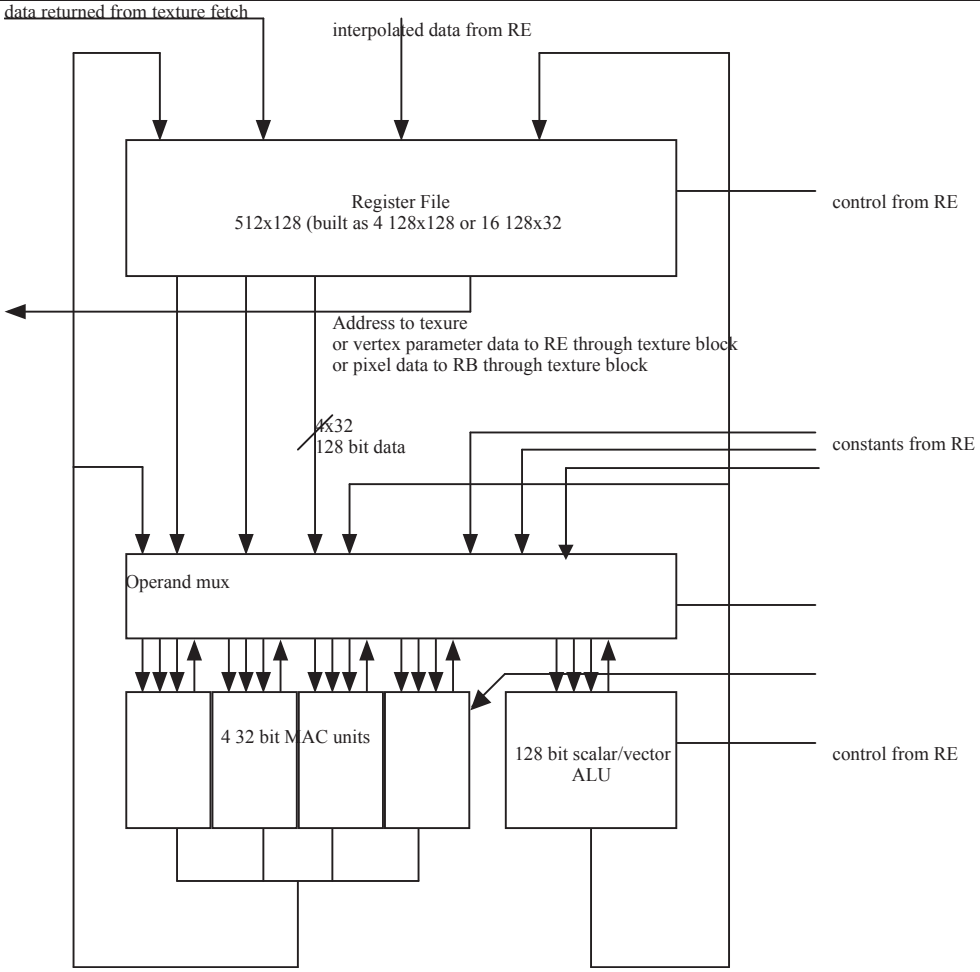
Uppon recept of the return data (identified by the tcmd containing the level number 0), the level 0 texture machine issues a register address for the return value (td). Then, it puts the finished packet in FIFO 1.

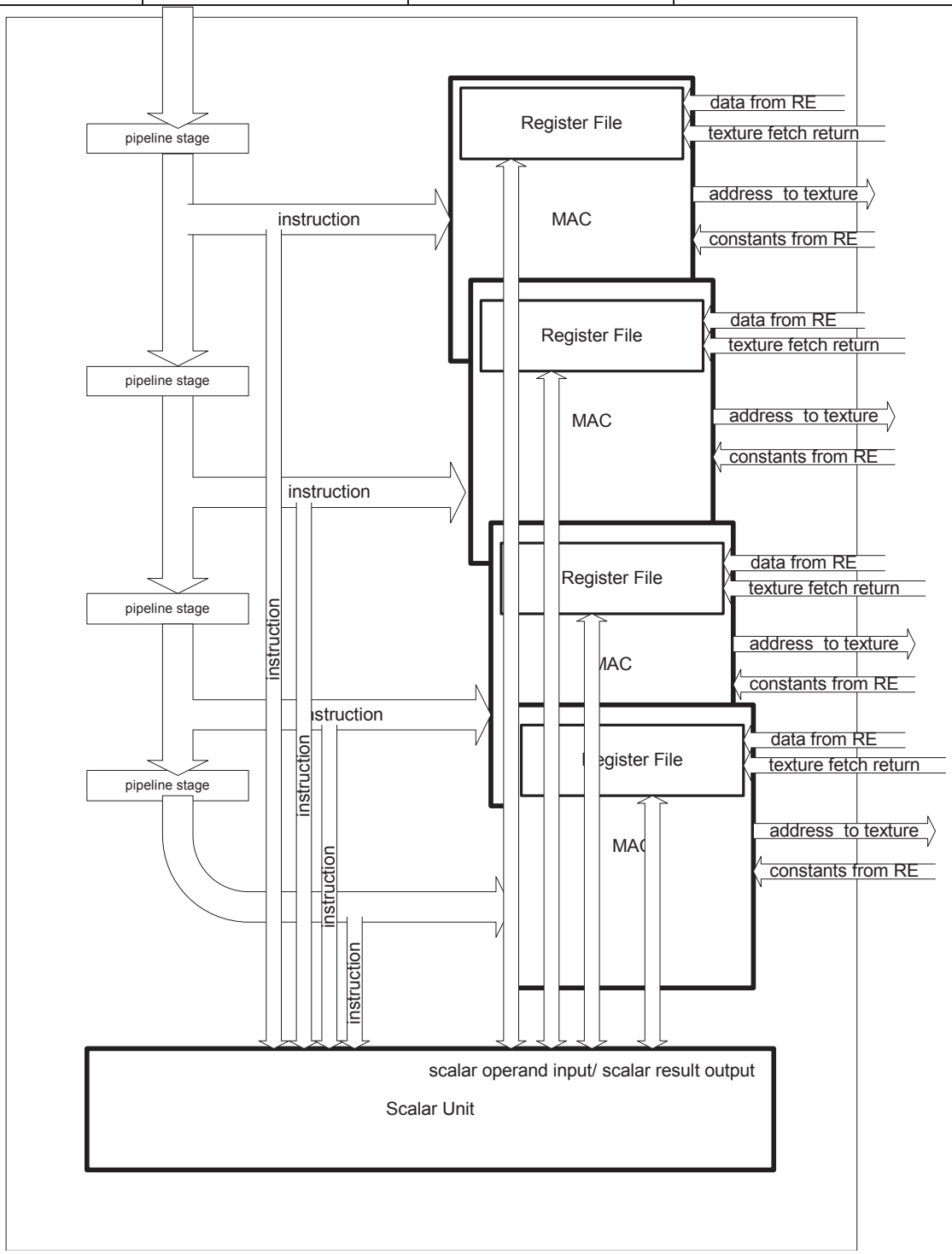
On receipt of a command, the level 0 ALU machine issues a complete set of level 0 shader instructions. For each instruction, the state machine generates 3 source addresses, one destination address (2 cycles later) and an instruction id wich is used to index into the instruction store. Once the last instruction as been issued, the packet is put into FIFO 2. Note that in the case of a pixel packet, the two vectors of 16 pixels are interleaved in order to hide the latency of the ALUs (8 cycles).

All other level process in the same way until the packet finally reaches the last ALU machine (8). On completion of the level 8 ALU clause, a valid bit is sent to the Render Backend wich picks up the color data. This requires that the last instruction writes to the output register – a condition that is almost always true. If the packet was a vertex packet, instead of sending the valid bit to the RB, it is sent to the PA, which picks up the data a puts it into the vertex store.

Only one ALU state machine may have access to the SRAM address bus or the instruction decode bus at one time. Similarly, only one texture state machine may have access to the SRAM address bus at one time. Arbitration is performed by two arbitrer blocks (one for the ALU state machines and one for the texture state machines). The arbitrers always favor the higher number state machines, preventing a bunch of half finished jobs from clogging up the SRAMS.

Each state machine maintains an address pointer specifying where the 16 (or 32) entries vector is located in the SRAM (the texture machine has two pointers one for the read address and one for the write). Upon completion of its job, the address pointer is incremented by a predefined amount equal to the total number of registers required by the shading code. A comparison of the address pointer for the first state machine in the chain (the input state machine), and the last machine in the chain (the level 8 ALU machine), gives an indication of how much unallocated SRAM memory is available. When this number falls below a preset watermark, the input state machine will stall the rasterizer preventing new data from entering the chain.


data returned from texture fetch
interpolated data from RE
Register File
512x128 (built as 4 128x128 or 16 128x32
control from RE
Address to texure
or vertex parameter data to RE through texture block
or pixel data to RB through texture block
4x32
128 bit data
constants from RE
Operand mux
4 32 bit MAC units
128 bit scalar/vector ALU
control from RE



2. Texture Arbitration

The texture arbitration logic chooses one of the 8 potentially pending texture clauses to be executed. The choice is made by looking at the fifos from 8 to 0 and picking the first one ready to execute. Once chosen, the clause state machine will send one 2x2 texture fetch per 4 clocks until all the texture fetch instructions of the clause are sent. This means that there cannot be any dependencies between two texture fetches of the same clause.

3. ALU Arbitration

ALU arbitration proceeds in almost the same way than texture arbitration. The ALU arbitration logic chooses one of the 8 potentially pending ALU clauses to be executed. The choice is made by looking at the fifos from 8 to 0 and picking the first one ready to execute. If the packet chosen is a packet of vertices, the state machine issues one instruction every 4 clocks until the clause is finished. This means that the compiler has to insert nops between two dependent successive instructions. If the packet is a pixel packet it is made out of two sub-vectors of 16. Thus the state machine issues the first instruction for the first sub-vector and then, 4 clocks later, the first instruction of the second sub-vector and so on until the clause is finished. Proceeding this way hides the latency of 8 clocks of the ALUs.

4. Input Interface

4.1 Rasterizer to Register File (interpolated data)

Name	Direction	bits	Description
SND	SEQ→SP	1	High when sending data
Interpolated data	SEQ→SP	512	512 bits transferred every 4 cycles

4.2 Texture Unit to Register File (texture return)

Name	Direction	bits	Description
SND	SEQ→TU	1	High when sending data
Texture colors	TU→SP	512	512 bits transferred every 4 cycles

4.3 ALU Unit to Register File (ALU op result)

Name	Direction	bits	Description
SND	SEQ→SP	1	High when sending data
Blend result ALU	SP→SP	512	512 bits transferred every 4 cycles
Write Mask	SP→SP	16	The four write masks

4.4 Scalar Unit to Register File (Scalar op result)

Name	Direction	bits	Description
SND	SEQ→SP	1	High when sending data
Scalar result	SP→SP	512	512 bits transferred every 4 cycles
Write Mask	SP→SP	16	The four write masks

5. Output Interface

5.1 Sequencer to Shader Engine Bus

This is a bus that sends the instruction and constant data to all 4 Sub-Engines of the Shader. Because a new instruction is needed only every 4 clocks, the width of the bus is divided by 4 and both constants and instruction are sent over those 4 clocks.

Name	Direction	Bits	Description
Instruction Start	SEQ-> SP	1	High on first cycle of transfer
Constant 0	SEQ-> SP	32	128 bits transferred over 4 cycles, alpha first…blue last
Constant 1	SEQ-> SP	32	128 bits transferred over 4 cycles, alpha first…blue last
Instruction	SEQ-> SP	40	160 bits transferred over 4 cycles

5.2 Shader Engine to Texture Unit Bus

One quad's worth of addresses is transferred to Texture Unit every clock. These are sourced fro a different pixel within each of the sub-engines repeating every 4 clocks. The register file index to read must precede the data by 2 clocks. The Read address associated with Quad 0 must be sent 1 clock after the Instruction Start signal is sent, so that data is read 3 clocks after the Instruction Start.

One Quad's worth of Texture Data may be written to the Register File every clock. These are directed to a different pixel of the sub-engines repeating every 4 clocks. The register file index to write must accompany the data. Data and Index associated with the Quad 0 must be sent 3 clocks after the Instruction Start signal is sent.

Name	Direction	Bits	Description
Tex_Read_Register_Index	SEQ->SP	8	Index into Register Files for reading Texture Address
Tex_RegFile_Read_Data	SP->TEX	512	4 Texture Addresses read from the Register File
Tex_Write_Register_Index	SEQ->SP	8	Index into Register file for write of returned Texture Data

6. Open issues

There is currently an issue with constants. If the constants are not the same for the whole vector of vertices, we don't have the bandwith from the texture store to feed the ALUs. Two solutions exists for this problem:

1) Let the compiler handle the case and put those instructions in a texture clause so we can use the bandwith there to operate. This requires a significant amount of temporary storeage in the register store.
2) Waterfall down the pipe allowing only at a given time the vertices having the same constants to operate in parralel. This might in the worst case slow us down by a factor of 16.